

THE CLAUDE BEAST

Claude Worktrees

Parallel Isolated Development Sessions via Git Worktrees

--worktree flag | Auto-cleanup | CLI + Desktop + IDE

What Are Claude Worktrees?

Multiple working directories from one repository -- each with its own branch and files

Core Concept

Git worktrees create multiple working directories that share the same .git history. Claude Code wraps this with `--worktree (-w)` for one-command creation, automatic branching, session isolation, and cleanup on exit.

Usage

```
# Named worktree
claude --worktree feature-auth

# Auto-generated name
claude --worktree # or: claude -w

# During a session
"work in a worktree"
```

Under the Hood

- Creates `.claude/worktrees/<name>/`
- Branches from default remote branch
- Branch named `worktree-<name>`
- Full file copy, shared .git history
- Auto-cleanup on session exit

Add `.claude/worktrees/` to your `.gitignore` to prevent worktree contents from appearing as untracked files

The Isolation Model

What is shared vs. what is separate across worktrees

Shared (via single .git)

- Full repository history (all commits, all branches)
- Remote connections (origin, upstream)
- Git configuration (.gitconfig, hooks)
- Refs and objects database

Separate (per worktree)

- Working directory (own copy of all files)
- Checked-out branch (worktree-<name>)
- Index / staging area
- Uncommitted changes
- Claude Code session context

Cleanup Behavior on Exit

No changes made

Worktree + branch removed automatically

Changes or commits exist

Claude prompts: keep or remove

High-Impact Use Cases



Parallel Feature + Bugfix

1

Work on a new feature in one worktree while fixing a bug in another.
No stashing, no branch switching -- true parallel development.



Three-Agent Pattern

2

One agent writes code, one writes tests, one reviews -- all in separate worktrees.
Eliminates sequential bottlenecks in the code-test-review cycle.



Competing Debugging Hypotheses

3

Test different theories in parallel and converge faster.
Each hypothesis gets its own isolated environment.



Long-Running Refactors

4

Start a worktree for a time-consuming refactor while continuing other work.
The main tree stays clean for other tasks.

Integration: Sub-Agents & Agent Teams

SUBAGENT WORKTREE ISOLATION

Agent Frontmatter

```
---
name: parallel-worker
description: Isolated parallel work
isolation: worktree
---
```

- Each subagent gets its own worktree
- File changes don't conflict with main session
- Auto-cleanup if no changes made
- Natural language: "use worktrees for your agents"

Guidance

Worktrees for manual parallel sessions (you orchestrate). Agent Teams for automated parallel coordination (Claude orchestrates).

WORKTREES VS AGENT TEAMS

Aspect	Worktrees	Agent Teams
Scope	File isolation	Full coordination
Communication	Results to parent	Peer-to-peer
Orchestration	You manage	Claude manages
Best for	Parallel editing	Complex collaboration

Practical Workflow: Three Terminals

Three parallel Claude sessions, each working independently on different tasks



Terminal 1: Feature

```
claude --worktree feature-  
auth
```

Working on authentication
refactor in isolated branch
worktree-feature-auth



Terminal 2: Bugfix

```
claude --worktree bugfix-123
```

Fixing critical bug in
isolated branch
worktree-bugfix-123



Terminal 3: Refactor

```
claude --worktree refactor-db
```

Database refactoring in
isolated branch
worktree-refactor-db

When Done: Standard Git Merge



Each worktree produces commits on its own branch. Merge via standard Git workflows: PR, merge, or cherry-pick. No special tooling needed.

Known Limitations & Tips

KNOWN LIMITATIONS

- `.claude/` subdirectories (skills, agents, docs) not auto-copied to worktrees
- Each worktree needs its own npm install / dependency setup
- Disk space: each worktree is a full copy of working files
- Git locks: some operations may conflict across worktrees
- No shared Claude session context between worktrees

BEST PRACTICES

- Add `.claude/worktrees/` to `.gitignore`
- **Use descriptive names** — feature-auth, not wt-1
- **Close worktrees when done** — avoid stale branches
- Run dependency install after creating worktree
- Use worktrees for file isolation, agent teams for coordination
- **Keep worktree count manageable** — 2-4 simultaneous is ideal



Known Issue (GitHub #28041)

`.claude/` subdirectories (skills, agents, docs, rules) are not automatically copied into worktrees. Only the worktree's own `.claude/` directory is created. Tracked as a bug -- expected to be fixed.

Worktrees in the Claude Code Ecosystem

The Parallelism Progression



Level 0

Solo Session

One Claude, one branch, one task



Level 1

Sub-Agents

Parallel workers, results flow back to parent



Level 2

Worktrees

Parallel isolated sessions, file-level isolation



Level 3

Agent Teams

Coordinated multi-agent with peer messaging



Key Insight

Worktrees are the file-isolation layer that makes all other parallelism safe. Sub-agents and agent teams both use worktrees under the hood for conflict-free editing.

Why Worktrees Matter

Elimination of Context Switching

No more stash-switch-unstash. Each task gets a persistent isolated workspace. True parallel development with zero collision risk.

The Foundation of Parallelism

Worktrees provide the file-isolation layer that makes sub-agents and agent teams safe. They are the enabling infrastructure for multi-agent development.

One-Command Simplicity

`claude --worktree feature-name`. That's it. Auto-branching, auto-cleanup, available across CLI, Desktop, IDE, web, and mobile.

`claude --worktree` | Available since February 2026 | Built on `git worktree`